Strings — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

80. Remove Outermost Parentheses

Understanding the Problem NEW

A 'primitive' valid parentheses string is one that doesn't split into smaller valid pieces on its own (like `(()())` or `()`). You're given a string made of several primitive pieces glued together, e.g. `(()())(())`, which splits into two primitive pieces: `(()())` and `(())`. Removing the outermost bracket from each piece leaves `()()` and `()`, so the final answer is `()()()`. The point: every primitive chunk has exactly one outermost opening and closing bracket that should be dropped, keeping everything inside untouched.

Brute Force Approach NEW

A beginner might try to first locate every primitive substring explicitly (by matching each opening bracket with its exact partner using a stack), slice out the substring, strip its first and last characters, and concatenate the results — correct, but more bookkeeping than necessary.

Why We Optimize NEW

You don't need to find and slice substrings at all — you just need to track your current **nesting depth** with a single counter as you scan character by character. Any `(` is kept only if depth is currently > 0 *before* incrementing (i.e. it's not an outermost opening bracket), and any `)` is kept only if depth > 1 *before* decrementing. This lets you build the answer in a single O(n) left-to-right pass with O(1) extra bookkeeping (just an integer counter), instead of explicitly identifying and slicing substrings.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove the outermost parentheses from every primitive valid parentheses string.

Visual Diagram

```
Input: "(()())(())"
       Primitive decompositions:
       (()()) → ( ) removed → ()()
       (())   → ( ) removed → ()
       Result: "()()()"
```

```
Step-by-step:
String: ( ( ) ( ) ) ( ( ) )
Index: 0 1 2 3 4 5 6 7 8 9

Initialize count=0, result=""
i=0: '(' → count=1 (skip first)
i=1: '(' → count=2, add '('
i=2: ')' → count=1, add ')'
i=3: '(' → count=2, add '('
i=4: ')' → count=1, add ')'
i=5: ')' → count=0 (skip last)
i=6: '(' → count=1 (skip first)
i=7: '(' → count=2, add '('
i=8: ')' → count=1, add ')'
i=9: ')' → count=0 (skip last)

Result: "()()()"
```

Solution Approach

Concept: Use counter to track depth, skip outermost parentheses.

Algorithm

```
Algorithm REMOVE_OUTER_PARENS(s):
           result = ""
           count = 0

           For each char c in s:
               If c == '(':
                    If count > 0: result += c

                   count++
              Else: // c == ')'
                   count--
                   If count > 0: result += c

          Return result
```

Java Implementation

```
class Solution {
          public String removeOuterParentheses(String s) {
              StringBuilder result = new StringBuilder();
              int count = 0;

              for (char c : s.toCharArray()) {
                  if (c == '(') {
                      if (count > 0) {
```

```
                        result.append(c);
                    }
                    count++;
                } else { // c == ')'
                    count--;
                    if (count > 0) {
                        result.append(c);
                    }
                }
            }
            return result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

81. Reverse Words in a Given String

Understanding the Problem NEW

Given a sentence, reverse the order of the words (not the letters within each word) while collapsing any extra spaces. E.g. `"the sky is blue"` → `"blue is sky the"`.

Brute Force Approach NEW

Manually scanning character by character to identify word boundaries and rebuild the reversed order in-place is fiddly and error-prone, especially handling multiple spaces, leading/trailing spaces, etc.

Why We Optimize NEW

The clean approach: split the string into a list of words (which naturally handles multiple spaces if you use a split that collapses whitespace), reverse that list, and join it back together with single spaces. This isn't really a 'brute vs optimal complexity' story so much as 'use the right built-in operations' — splitting and reversing a list of words is O(n) overall and far less error-prone than manual in-place character shuffling.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse the order of words in a string.

Visual Diagram

```
Input: "the sky is blue"
       Output: "blue is sky the"

       Step-by-step:
       1. Split into words: ["the", "sky", "is", "blue"]
       2. Reverse array: ["blue", "is", "sky", "the"]
       3. Join with spaces: "blue is sky the"

       Input: " hello world     "
       Output: "world hello"

       Step-by-step:
```

```
1. Trim spaces and split: ["hello", "world"]
2. Reverse: ["world", "hello"]
3. Join: "world hello"
```

Solution Approach

Concept: Split by spaces, reverse, and join.

Algorithm

```
Algorithm REVERSE_WORDS(s):
           words = split(s.trim())
           reverse(words)
           Return join(words, " ")
```

Java Implementation

```
class Solution {
           public String reverseWords(String s) {
               // Split by spaces and remove empty strings
               String[] words = s.trim().split("\\s+");

                 // Reverse words
                 StringBuilder result = new StringBuilder();
                 for (int i = words.length - 1; i >= 0; i--) {
                     result.append(words[i]);
                     if (i > 0) {
                         result.append(" ");
                     }
                 }
                 return result.toString();
           }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

82. Largest Odd Number in a String

Understanding the Problem NEW

Given a string of digits, find the largest possible number formed by taking some prefix of the string, with the requirement that the resulting number is odd. E.g. `"52"` → the only odd-ending prefix option is none... consider `"35427"` → largest odd prefix is `"35427"` itself since it ends in 7 (odd); but `"4206"` has no odd digit at all so the answer is empty.

Brute Force Approach NEW

A beginner might generate every possible prefix of the string, convert each to a number, check if it's odd, and track the largest — expensive if done via actual numeric conversion (numbers can be huge) and unnecessary work.

Why We Optimize NEW

The 'largest' prefix is always the *longest* valid one (since more digits at the front always makes a bigger number, assuming no leading-zero edge cases to worry about here). So you just need to find the **rightmost digit that is odd**, scanning from the end of the string backward, and take everything up to and including that digit. One O(n) backward scan, no numeric conversion or prefix enumeration needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the largest odd number in a string.

Visual Diagram

```
Input: "35427"
       Output: "35427" (all digits form odd number)

       Check from right:
       '7' is odd → entire string "35427" is odd
       Result: "35427"

       Input: "4206"
       Output: "" (no odd number)
```

```
Check from right:
'6' even, '0' even, '2' even, '4' even
No odd found → return ""

Input: "52"
Output: "5"

Check from right:
'2' even
'5' odd → substring "5"
Result: "5"
```

Solution Approach

Concept: Find rightmost odd digit.

Algorithm

```
Algorithm LARGEST_ODD(s):
           For i = s.length-1 down to 0:
               If (s[i] - '0') % 2 == 1:
                   Return s.substring(0, i+1)
           Return ""
```

Java Implementation

```
class Solution {
           public String largestOddNumber(String num) {
               for (int i = num.length() - 1; i >= 0; i--) {
                   if ((num.charAt(i) - '0') % 2 == 1) {
                        return num.substring(0, i + 1);
                   }
               }
               return "";
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

83. Longest Common Prefix

Understanding the Problem NEW

Given an array of strings, find the longest string that is a prefix of *every* string in the array. E.g. `["flower","flow","flight"]` → common prefix is `"fl"`.

Brute Force Approach NEW

Compare every string against every other string letter by letter to find matches — this can be done but is often written inefficiently, e.g. re-comparing pairs of strings repeatedly rather than tracking one shrinking answer.

Why We Optimize NEW

A clean way to think about it: start by assuming the *first* string entirely is the common prefix, then walk through the **rest** of the strings one at a time, shrinking your candidate prefix down every time it doesn't match the current string, until either the candidate becomes empty (no common prefix) or you've checked every string. This visits each character a bounded number of times overall, giving roughly O(sum of string lengths) — you're incrementally narrowing one answer rather than comparing every pair separately.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest common prefix among strings.

Visual Diagram

```
Input: ["flower", "flow", "flight"]
       Output: "fl"

       Step 1: Compare "flower" and "flow"
       Common: "flow"
       Step 2: Compare "flow" and "flight"
       Common: "fl"
       Result: "fl"

       Input: ["dog", "racecar", "car"]
       Output: ""
```

```
       Step 1: Compare "dog" and "racecar"
       Common: ""
       Result: ""

       Horizontal Scanning:
       Take first word "flower"
       Compare with "flow": common "flow"
       Compare with "flight": common "fl"
       Return "fl"
```

Solution Approach

Concept: Compare first string with all others.

Algorithm

```
Algorithm LONGEST_COMMON_PREFIX(strs):
           If empty: Return ""
           prefix = strs[0]
           For i = 1 to n-1:
               While strs[i].indexOf(prefix) != 0:
                   prefix = prefix.substring(0, prefix.length-1)
                   If prefix == "": Return ""
           Return prefix
```

Java Implementation

```
class Solution {
           public String longestCommonPrefix(String[] strs) {
               if (strs.length == 0) return "";

                 String prefix = strs[0];

                 for (int i = 1; i < strs.length; i++) {
                     while (strs[i].indexOf(prefix) != 0) {
                         prefix = prefix.substring(0, prefix.length() - 1);
                         if (prefix.isEmpty()) return "";
                     }
                 }
                 return prefix;
            }
       }
```

Time & Space Complexity

Time Complexity: O(n * m) | Space Complexity: O(1)

84. Isomorphic String

Understanding the Problem NEW

Two strings are 'isomorphic' if you can consistently replace every character in one string with a corresponding character in the other, such that the pattern of repetition matches exactly. E.g. `"egg"` and `"add"` are isomorphic (e→a, g→d) but `"foo"` and `"bar"` are not.

Brute Force Approach NEW

A beginner might try building a mapping from the first string's characters to the second's, but forget to check the mapping is **consistent in both directions** — e.g. two different characters in string 1 both mapping to the same character in string 2 would wrongly be accepted without a check.

Why We Optimize NEW

You need **two** hashmaps, not one: one mapping characters from string 1 → string 2, and another mapping string 2 → string 1. At each position, you check that both mappings agree with what's already been recorded (or create a new mapping entry if this character pair hasn't been seen yet). This bidirectional check is what correctly enforces a strict one-to-one correspondence, still in a single O(n) pass with O(1) extra space (since there are a bounded number of possible characters).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two strings are isomorphic (characters can be mapped one-to-one).

Visual Diagram

```
Input: "egg", "add"
       Output: true

       Mapping: e→a, g→d
       "egg" → "add" ✓

       Input: "foo", "bar"
       Output: false

       Mapping: f→b, o→a, but o→r (conflict)
```

```
        "foo" → "bar" ✗

        Input: "paper", "title"
        Output: true

        Mapping: p→t, a→i, p→t (consistent), e→l, r→e
        "paper" → "title" ✓

        Two maps approach:
        Map1: char in s → char in t
        Map2: char in t → char in s

        For "egg", "add":
        e→a, a→e
        g→d, d→g
        Both maps consistent ✓
```

Solution Approach

Concept: Use two maps for bidirectional mapping.

Algorithm

```
Algorithm IS_ISOMORPHIC(s, t):
            If length different: Return false
            map_s = {}, map_t = {}
            For i = 0 to n-1:
                char_s = s[i], char_t = t[i]
                If map_s contains char_s:
                    If map_s[char_s] != char_t: Return false
                Else:
                    If map_t contains char_t: Return false
                    map_s[char_s] = char_t
                    map_t[char_t] = char_s
            Return true
```

Java Implementation

```
class Solution {
            public boolean isIsomorphic(String s, String t) {
                if (s.length() != t.length()) return false;

                  Map<Character, Character> mapS = new HashMap<>();
                  Map<Character, Character> mapT = new HashMap<>();
                  for (int i = 0; i < s.length(); i++) {

                    char c1 = s.charAt(i);
                    char c2 = t.charAt(i);

                    if (mapS.containsKey(c1)) {
```

```
                    if (mapS.get(c1) != c2) return false;
                } else {
                    if (mapT.containsKey(c2)) return false;
                    mapS.put(c1, c2);
                    mapT.put(c2, c1);
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1) (max 256 characters)

85. Check Whether One String is a Rotation of Another

Understanding the Problem NEW

Check whether string B can be obtained by rotating string A some number of positions. E.g. `"waterbottle"` rotated becomes things like `"erbottlewat"` — is a given target a valid rotation?

Brute Force Approach NEW

A beginner might literally generate every possible rotation of A (shifting one character at a time and rebuilding the string) and compare each one to B — O(n) rotations, each requiring an O(n) string comparison, giving $O(n^2)$ total.

Why We Optimize NEW

Here's a neat trick: if you concatenate A with itself (`A+A`), every possible rotation of A appears somewhere inside that doubled string as a contiguous substring. So the whole problem reduces to a single substring-search: is B a substring of `A+A` ? Using an efficient substring search algorithm, this check can be done in roughly O(n) time instead of generating and comparing every rotation individually — the doubling trick avoids explicit rotation entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if one string is a rotation of another.

Visual Diagram

```
Input: s1 = "abcde", s2 = "cdeab"
       Output: true

       s1 = "abcde"
       Possible rotations:
       "abcde"
       "bcdea"
       "cdeab" ✓
       "deabc"
       "eabcd"
```

```
Approach: Check if s2 is substring of s1+s1
s1 + s1 = "abcdeabcde"
"cdeab" is substring ✓

Input: s1 = "abcde", s2 = "abced"
Output: false

s1 + s1 = "abcdeabcde"
"abced" not substring ✗
```

Solution Approach

Concept: Check if s2 is substring of s1+s1.

Algorithm

```
Algorithm IS_ROTATION(s1, s2):
          If length different: Return false
          Return (s1 + s1).contains(s2)
```

Java Implementation

```
class Solution {
          public boolean isRotation(String s1, String s2) {
              if (s1.length() != s2.length()) return false;
              return (s1 + s1).contains(s2);
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) (contains uses KMP internally) | Space Complexity: O(n)

86. Check if Two Strings are Anagrams

Understanding the Problem NEW

Check whether two strings are anagrams of each other — meaning they contain exactly the same characters with the same frequency, just possibly in a different order. E.g. `"listen"` and `"silent"`.

Brute Force Approach NEW

A beginner approach: sort both strings alphabetically and check if the sorted versions are equal. This works, but sorting costs O(n log n) when something cheaper is available.

Why We Optimize NEW

Since you only care about *frequency*, not order, you can build a **frequency count** (an array of size 26 for lowercase letters, or a hashmap in general) for each string in O(n), then compare the two frequency counts directly in O(26) or O(alphabet size). This avoids sorting entirely, dropping the total from O(n log n) down to O(n) — trading 'sort and compare' for 'count and compare,' which is cheaper because comparing fixed-size frequency tables is much faster than sorting variable-length strings.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if two strings are anagrams of each other.

Visual Diagram

```
Input: s = "anagram", t = "nagaram"
       Output: true

       Frequency count:
       a: 3 (s) → 3 (t)
       n: 1 → 1
       g: 1 → 1
       r: 1 → 1
       m: 1 → 1
       All match ✓
```

```
Input: s = "rat", t = "car"
Output: false

Frequency count:
r: 1 (s) → 0 (t) ✗
a: 1 → 1
t: 1 → 1
r mismatch ✗
```

Solution Approach

Concept: Use character frequency array.

Algorithm

```
Algorithm IS_ANAGRAM(s, t):
           If length different: Return false
           freq = array of size 26

           For each char in s: freq[char-'a']++
           For each char in t: freq[char-'a']--

           For each count in freq:
               If count != 0: Return false
           Return true
```

Java Implementation

```
class Solution {
           public boolean isAnagram(String s, String t) {
               if (s.length() != t.length()) return false;

                 int[] freq = new int[26];

                 for (char c : s.toCharArray()) {
                     freq[c - 'a']++;
                 }

                 for (char c : t.toCharArray()) {
                     freq[c - 'a']--;
                     if (freq[c - 'a'] < 0) return false;
                 }

                 return true;

           }
     }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

87. Sort Characters by Frequency

Understanding the Problem NEW

Given a string, rearrange its characters so they appear in order of decreasing frequency — the most common character first, repeated as many times as it appears, then the next most common, and so on. E.g. `"tree"` → `"eert"` or `"eetr"`.

Brute Force Approach NEW

A beginner might repeatedly scan the whole remaining string to find the currently most frequent character, append it, and remove it — repeating this scan-and-remove process character by character, which is O(n²) since each scan costs O(n) and you do it up to n times.

Why We Optimize NEW

Instead, count all character frequencies once in O(n) using a hashmap, then **sort the distinct characters** by their frequency (descending) — O(k log k) where k is the number of distinct characters (at most 26 for lowercase letters, so effectively constant). Finally, build the output string by repeating each character according to its count. This turns repeated O(n) rescans into a single count pass plus one small sort, giving roughly O(n + k log k) instead of O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort characters by frequency in descending order.

Visual Diagram

```
Input: "tree"
       Output: "eert" or "eetr"

       Frequency:
       'e': 2
       't': 1
       'r': 1
       Sort by frequency: e(2), t(1), r(1)

       Input: "cccaaa"
```

```
Output: "cccaaa" or "aaaccc"

Frequency:
'c': 3
'a': 3
Both frequency 3 → either order

Input: "Aabb"
Output: "bbAa" or "bbaA"

Frequency:
'b': 2
'A': 1
'a': 1
Sort: bb then Aa
```

Solution Approach

Concept: Count frequencies and sort.

Algorithm

```
Algorithm SORT_BY_FREQUENCY(s):
           freq = array of size 256
           For each char in s: freq[char]++

           Sort characters by frequency desc
           Build result by repeating characters
           Return result
```

Java Implementation

```
class Solution {
           public String frequencySort(String s) {
               Map<Character, Integer> freq = new HashMap<>();
               for (char c : s.toCharArray()) {
                   freq.put(c, freq.getOrDefault(c, 0) + 1);
               }

                 // Sort by frequency
                 List<Character> chars = new ArrayList<>(freq.keySet());
                 chars.sort((a, b) -> freq.get(b) - freq.get(a));

               StringBuilder result = new StringBuilder();
               for (char c : chars) {
                   int count = freq.get(c);
                   for (int i = 0; i < count; i++) {
                       result.append(c);
                   }
               }
```

```
            return result.toString();
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

88. Maximum Nesting Depth of Parentheses

Understanding the Problem NEW

Given a string of matched parentheses (possibly with other characters), find the maximum nesting depth — how many levels of parentheses are open at the deepest point. E.g. `"(1+(2*3)+((8)/4))"` has max depth 3.

Brute Force Approach NEW

Without a plan, a beginner might try to explicitly identify matching bracket pairs first (e.g. using a stack to record positions) and then compute nesting levels from those pairs — more machinery than the problem actually needs.

Why We Optimize NEW

You don't need to match pairs at all — just keep a single running counter: increment it on every `(`, decrement on every `)`, and track the maximum value the counter ever reaches. The maximum value the counter hits *is* the maximum nesting depth, computed in a single O(n) left-to-right pass with O(1) space — no stack of positions or explicit pair-matching required, since you only care about the depth number, not which brackets matched which.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum nesting depth of parentheses in a string.

Visual Diagram

```
Input: "(1+(2*3)+((8)/4))+1"
       Output: 3

       Trace:
       String: ( 1 + ( 2 * 3 ) + ( ( 8 ) / 4 ) ) + 1
       Depth: 1 1 2 2 2 2 1 2 3 3 2 2 1 1 1 0

       Maximum depth = 3

       Input: "(1)+((2))+(((3)))"
       Output: 3
```

```
Trace:
(1)    → depth 1
((2)) → depth 2
(((3))) → depth 3
Maximum = 3
```

Solution Approach

Concept: Track current depth with a counter.

Algorithm

```
Algorithm MAX_DEPTH(s):
           max_depth = 0
           current_depth = 0

           For each char in s:
               If char == '(':
                   current_depth++
                   max_depth = max(max_depth, current_depth)
               Else If char == ')':
                   current_depth--

           Return max_depth
```

Java Implementation

```
class Solution {
           public int maxDepth(String s) {
               int maxDepth = 0;
               int currentDepth = 0;

                 for (char c : s.toCharArray()) {
                     if (c == '(') {
                         currentDepth++;
                         maxDepth = Math.max(maxDepth, currentDepth);
                     } else if (c == ')') {
                         currentDepth--;
                     }
                 }
                 return maxDepth;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

89. Roman Number to Integer and Vice Versa

Understanding the Problem NEW

Convert between Roman numerals (like `"MCMXCIV"`) and integers (1994), in both directions.

Brute Force Approach NEW

For integer-to-Roman, a beginner might try to handle every possible digit combination with a giant set of if/else conditions per digit position — workable but repetitive and error-prone. For Roman-to-integer, a common mistake is to just add up each symbol's value without accounting for subtractive cases like `IV` (4, not 6).

Why We Optimize NEW

Both directions have a clean **greedy** solution. For integer→Roman: keep a table of value/symbol pairs sorted from largest to smallest (including subtractive combinations like 'CM'=900, 'IV'=4), and repeatedly subtract the largest value that fits, appending its symbol, until the number reaches 0 — this greedily builds the correct Roman numeral without per-digit special-casing. For Roman→integer: scan left to right, and if a symbol's value is smaller than the value that comes right after it, subtract it instead of adding it (this correctly captures subtractive notation like IV, IX, XL, etc.). Both run in O(1) time in practice (Roman numerals are bounded in length) and avoid a giant hand-written case explosion.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert Roman numerals to integer and vice versa.

Visual Diagram

```
Roman to Integer:
       Input: "MCMXCIV"
       Output: 1994

       M = 1000
       CM = 900 (C=100 before M=1000)
       XC = 90 (X=10 before C=100)
```

```
IV = 4 (I=1 before V=5)
Total = 1000 + 900 + 90 + 4 = 1994

Rules:
I(1), V(5), X(10), L(50), C(100), D(500), M(1000)
Smaller before larger = subtract

Integer to Roman:
Input: 1994
Output: "MCMXCIV"

1994 = 1000 + 900 + 90 + 4
1000 → M
900 → CM
90 → XC
4 → IV
Result: MCMXCIV
```

Solution Approach

Concept: Greedy approach for both conversions.

Algorithm (Roman to Integer)

Algorithm ROMAN_TO_INT(s):

map = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}

result = 0

For i = 0 to n-1:

If i+1 < n and map[s[i]] < map[s[i+1]]:

result -= map[s[i]]

Else:

result += map[s[i]]

Return result

Algorithm (Integer to Roman)

Algorithm INT_TO_ROMAN(num):

values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1]

symbols = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"]

result = ""

For i = 0 to 12:

While num >= values[i]:

result += symbols[i]

num -= values[i]

Return result

Java Implementation

```
class Solution {
            // Roman to Integer
            public int romanToInt(String s) {
                Map<Character, Integer> map = new HashMap<>();
                map.put('I', 1);
                map.put('V', 5);
                map.put('X', 10);
                map.put('L', 50);
                map.put('C', 100);
                map.put('D', 500);
                map.put('M', 1000);

                int result = 0;
                for (int i = 0; i < s.length(); i++) {
                    if (i + 1 < s.length() && map.get(s.charAt(i)) <
map.get(s.charAt(i + 1))) {
                        result -= map.get(s.charAt(i));
                    } else {
                        result += map.get(s.charAt(i));
                    }
                }
                return result;
            }

            // Integer to Roman
            public String intToRoman(int num) {
                int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4,
1};
                String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X",
"IX", "V", "IV", "

                StringBuilder result = new StringBuilder();
                for (int i = 0; i < values.length; i++) {
                    while (num >= values[i]) {
                        result.append(symbols[i]);
                        num -= values[i];
                    }
                }
                return result.toString();
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) for roman, O(1) for integer | Space Complexity: O(1)

90. Implement Atoi

Understanding the Problem NEW

Implement the classic `atoi` function: convert a string (which may have leading whitespace, an optional +/- sign, then digits, possibly followed by non-digit junk) into an integer, clamping to the valid 32-bit integer range if it overflows.

Brute Force Approach NEW

This isn't really a 'brute force vs optimized' problem — the real challenge is just being thorough about all the edge cases: leading whitespace, sign handling, stopping at the first non-digit character, and overflow/underflow clamping. A beginner's first attempt often forgets one or more of these (especially overflow, or strings that are entirely non-numeric).

Why We Optimize NEW

The reliable approach is to process the string in clearly separated stages, in order: (1) skip leading whitespace, (2) check for an optional sign, (3) consume digits one at a time while building up the number, and (4) at each digit, check *before* multiplying whether you're about to overflow the valid range, clamping and stopping early if so. Handling overflow proactively (checking before it happens) rather than after the fact (letting the number wrap around) is the key correctness detail here — this remains a simple O(n) single pass, but the 'optimization' is really about correctness robustness rather than speed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert string to integer (atof/atoi implementation).

Visual Diagram

```
Input: "   -42"
       Output: -42

       Steps:
       1. Skip leading spaces: "   -42" → "-42"
       2. Check sign: '-' → negative
       3. Parse digits: "42" → 42
```

```
4. Apply sign: -42
5. Check range: -42 within [-2^31, 2^31-1]

Input: "4193 with words"
Output: 4193

Steps:
1. Skip spaces: "4193 with words"
2. No sign
3. Parse digits: "4193" → 4193
4. Stop at non-digit space
5. Result: 4193

Input: "words and 987"
Output: 0

Steps:
1. Skip spaces: "words and 987"
2. No digits found → return 0

Input: "-91283472332"
Output: -2147483648

Steps:
1. Parse "-91283472332"
2. Less than Integer.MIN_VALUE
3. Return Integer.MIN_VALUE
```

Solution Approach

Concept: Handle whitespace, sign, digits, and overflow.

Algorithm

```
Algorithm MY_ATOI(s):
           i = 0, n = s.length()
           sign = 1, result = 0

            // Skip spaces
            While i < n and s[i] == ' ': i++

            // Handle sign
            If i < n and (s[i] == '+' or s[i] == '-'):
                sign = s[i] == '-' ? -1 : 1
                i++

            // Parse digits
            While i < n and s[i] >= '0' and s[i] <= '9':

              digit = s[i] - '0'
              // Check overflow
```

```
            If result > (Integer.MAX_VALUE - digit) / 10:
                Return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE
            result = result * 10 + digit
            i++

        Return result * sign
```

Java Implementation

```
class Solution {
        public int myAtoi(String s) {
            int i = 0, n = s.length();
            int sign = 1;
            long result = 0;

            // Skip leading whitespace
            while (i < n && s.charAt(i) == ' ') {
                i++;
            }

            // Handle sign
            if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
                sign = s.charAt(i) == '-' ? -1 : 1;
                i++;
            }

            // Parse digits
            while (i < n && s.charAt(i) >= '0' && s.charAt(i) <= '9') {
                int digit = s.charAt(i) - '0';
                result = result * 10 + digit;

                // Check overflow
                if (result * sign > Integer.MAX_VALUE) {
                     return Integer.MAX_VALUE;
                }
                if (result * sign < Integer.MIN_VALUE) {
                     return Integer.MIN_VALUE;
                }
                i++;
            }

            return (int)(result * sign);
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

91. Count Number of Substrings

Understanding the Problem NEW

Count the number of substrings that contain at least K distinct characters. This is a variation of many substring-counting problems, using a sliding window.

Brute Force Approach NEW

Check every possible substring (all start/end pairs), count its distinct characters, and tally the ones meeting the K threshold — $O(n^2)$ or $O(n^3)$ if you recompute distinct-character counts from scratch for every substring.

Why We Optimize NEW

Use a **sliding window with a frequency map**: as you expand the window's right edge, update the count of distinct characters seen; whenever the window has at least K distinct characters, every further expansion to the right still keeps that property, so you can count all valid substrings starting from the current left edge in O(1) rather than re-scanning. By shrinking from the left when needed and reusing the frequency map incrementally (rather than recomputing it per substring), the whole count can be computed in O(n) or O(n × alphabet size), instead of re-examining every substring independently.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count substrings with at least K distinct characters.

Visual Diagram

```
Input: s = "abc", K = 2
       Output: 3

       Substrings with ≥ 2 distinct:
       "ab" (a,b)
       "abc" (a,b,c)
       "bc" (b,c)
       Total: 3

       Input: s = "abac", K = 2
       Output: 7
```

```
All substrings with ≥ 2 distinct:
"ab", "aba", "abac", "ba", "bac", "ac", "abc"
Wait, "abc" not in string... Let's list carefully:

All substrings:
a: 1 distinct ✗
ab: 2 distinct ✓
aba: 2 distinct ✓
abac: 3 distinct ✓
b: 1 ✗
ba: 2 ✓
bac: 3 ✓
a: 1 ✗
ac: 2 ✓
c: 1 ✗
Total: 7
```

Solution Approach

Concept: Sliding window with frequency map.

Algorithm

```
Algorithm COUNT_SUBSTRINGS(s, K):
           count = 0
           For i = 0 to n-1:
               freq = {}
               distinct = 0
               For j = i to n-1:
                   If freq[s[j]] == 0: distinct++
                   freq[s[j]]++
                   If distinct >= K: count++
           Return count
```

Java Implementation

```
class Solution {
           public int countSubstrings(String s, int k) {
               int count = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     int[] freq = new int[26];

                   int distinct = 0;

                   for (int j = i; j < n; j++) {
                       int index = s.charAt(j) - 'a';
                       if (freq[index] == 0) distinct++;
```

```
                    freq[index]++;

                    if (distinct >= k) count++;
                }
            }
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

92. Longest Palindromic Substring

Understanding the Problem NEW

Find the longest substring of the given string that reads the same forwards and backwards (a palindrome). E.g. in `"babad"`, both `"bab"` and `"aba"` are valid answers (length 3).

Brute Force Approach NEW

The most direct approach: check every possible substring (all start/end pairs) to see if it's a palindrome — $O(n^2)$ substrings, and checking each one takes up to $O(n)$ time, giving $O(n^3)$ total.

Why We Optimize NEW

A palindrome is symmetric around its center, so instead of checking substrings from the outside in, you can pick every possible **center** (there are 2n-1 of them, since a center can be a single character for odd-length palindromes, or a gap between two characters for even-length ones) and **expand outward** from each center as long as the characters on both sides keep matching. Each expansion naturally stops as soon as symmetry breaks, and you track the longest one found. This still checks $O(n)$ centers, but each expansion only costs as much as the palindrome actually found (not a fixed $O(n)$ check regardless), bringing the total down to $O(n^2)$ instead of $O(n^3)$ — a smarter way of generating candidates rather than checking every possible substring blindly.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the longest palindromic substring in a string.

Visual Diagram

```
Input: "babad"
       Output: "bab" or "aba"

       Step 1: Check all substrings
       "b" → palindrome length 1
       "bab" → palindrome length 3 ✓
       "babad" → not palindrome
```

```
"aba" → palindrome length 3 ✓

Expanding around center:
Center: index 1 ('a')
Left=0 ('b'), Right=2 ('b') → "bab" ✓
Left=-1, Right=3 → stop
Max length = 3

Center: between index 1 and 2
Left=1 ('a'), Right=2 ('b') → no match
Max length = 1

Input: "cbbd"
Output: "bb"

Center between index 1 and 2:
Left=1 ('b'), Right=2 ('b') → "bb" ✓
Left=0 ('c'), Right=3 ('d') → no match
Max length = 2
```

Solution Approach

Concept: Expand around centers (2n-1 centers).

Algorithm

```
Algorithm LONGEST_PALINDROME(s):
           n = length(s)
           start = 0, max_len = 1

            For i = 0 to n-1:
                // Odd length palindrome
                left = i, right = i
                While left >= 0 and right < n and s[left] == s[right]:
                    If right - left + 1 > max_len:
                        start = left
                        max_len = right - left + 1
                    left--; right++

                 // Even length palindrome
                 left = i, right = i+1
                 While left >= 0 and right < n and s[left] == s[right]:
                     If right - left + 1 > max_len:
                         start = left
                         max_len = right - left + 1
                     left--; right++

            Return s.substring(start, start + max_len)
```

Java Implementation

```
class Solution {
           public String longestPalindrome(String s) {
               if (s == null || s.length() < 2) return s;

               int start = 0, maxLen = 1;
               int n = s.length();

               for (int i = 0; i < n; i++) {
                   // Odd length
                   int left = i, right = i;
                   while (left >= 0 && right < n && s.charAt(left) ==
s.charAt(right)) {
                       if (right - left + 1 > maxLen) {
                           start = left;
                           maxLen = right - left + 1;
                       }
                       left--;
                       right++;
                   }

                   // Even length
                   left = i;
                   right = i + 1;
                   while (left >= 0 && right < n && s.charAt(left) ==
s.charAt(right)) {
                       if (right - left + 1 > maxLen) {
                           start = left;
                           maxLen = right - left + 1;
                       }
                       left--;
                       right++;
                   }
               }

               return s.substring(start, start + maxLen);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

93. Sum of Beauty of All Substrings

Understanding the Problem NEW

For every substring of the given string, define its 'beauty' as (frequency of its most common character) minus (frequency of its least common character); sum this beauty value over every substring in the string.

Brute Force Approach NEW

Directly following the definition: for every substring, build a frequency count of its characters (O(n) per substring in the worst case), then compute max-freq minus min-freq — with O(n²) substrings, each costing up to O(n) to analyze, giving O(n³) total.

Why We Optimize NEW

You can avoid recomputing frequencies from scratch for every substring: fix a starting index, and as you extend the substring one character at a time to the right, **incrementally update** a running frequency count instead of rebuilding it — after each single-character extension, updating the max and min frequency values only needs an O(alphabet size) scan (since there are only up to 26 lowercase letters to check), not a full recount of the substring. This turns the per-substring cost from O(n) down to O(26), reducing the overall complexity from O(n³) to O(n² × 26), i.e. effectively O(n²).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate sum of beauty (max_freq - min_freq) for all substrings.

Visual Diagram

```
Input: "aabcb"
       Output: 5

       All substrings:
       "a": 1-1=0
       "aa": 2-2=0
       "aab": 2-1=1
       "aabc": 2-1=1
```

```
"aabcb": 2-1=1
"a": 1-1=0
"ab": 1-1=0
"abc": 1-1=0
"abcb": 2-1=1
"b": 1-1=0
"bc": 1-1=0
"bcb": 2-1=1
"c": 1-1=0
"cb": 1-1=0
"b": 1-1=0
Total: 1+1+1+1+1+1 = 5
```

Solution Approach

Concept: For each substring, track min and max frequency.

Algorithm

```
Algorithm SUM_OF_BEAUTY(s):
           total = 0
           For i = 0 to n-1:
               freq = array of size 26
               For j = i to n-1:
                   freq[s[j]-'a']++
                   min_freq = INF, max_freq = -INF
                   For each count in freq:
                        If count > 0:
                            min_freq = min(min_freq, count)
                            max_freq = max(max_freq, count)
                   total += max_freq - min_freq
           Return total
```

Java Implementation

```
class Solution {
           public int beautySum(String s) {
               int total = 0;
               int n = s.length();

                 for (int i = 0; i < n; i++) {
                     int[] freq = new int[26];

                    for (int j = i; j < n; j++) {
                        freq[s.charAt(j) - 'a']++;

                         int minFreq = Integer.MAX_VALUE;

                       int maxFreq = Integer.MIN_VALUE;
```

```
                    for (int count : freq) {
                        if (count > 0) {
                            minFreq = Math.min(minFreq, count);
                            maxFreq = Math.max(maxFreq, count);
                        }
                    }

                    total += maxFreq - minFreq;
                }
            }
            return total;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

94. Reverse Every Word in a String

Understanding the Problem NEW

Given a sentence, reverse the letters within each word individually, while keeping the words themselves in their original order (the opposite emphasis from problem 81, which reversed word order but not the letters). E.g. `"Let's take LeetCode contest" → "s'teL ekat edoCteeL tsetnoc"`.

Brute Force Approach NEW

A beginner might manually track index boundaries and reverse characters in-place with careful pointer arithmetic — doable, but easy to get off-by-one errors on word boundaries, especially with multiple spaces.

Why We Optimize NEW

This is cleanly handled by splitting the string into words (by spaces), reversing the *characters within* each word individually (leaving word order untouched), and joining them back together with single spaces. Contrasting this directly with problem 81 (which reverses word order but keeps letters intact) makes it easy to remember the distinction: 'reverse the list of words' vs 'reverse each word in the list' are two different operations on the same underlying split-word representation, and confusing the two is a common source of mistakes.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse each word in a string while preserving word order.

Visual Diagram

```
Input: "Let's take LeetCode contest"
       Output: "s'teL ekat edoCteeL tsetnoc"

       Step 1: Split words
       ["Let's", "take", "LeetCode", "contest"]

       Step 2: Reverse each word
       "Let's" → "s'teL"
       "take" → "ekat"
       "LeetCode" → "edoCteeL"
```

```
       "contest" → "tsetnoc"

       Step 3: Join with spaces
       "s'teL ekat edoCteeL tsetnoc"

       Input: "God Ding"
       Output: "doG gniD"

       "God" → "doG"
       "Ding" → "gniD"
       Result: "doG gniD"
```

Solution Approach

Concept: Split, reverse each, join.

Algorithm

```
Algorithm REVERSE_EACH_WORD(s):
           words = split(s, " ")
           For each word in words:
               Reverse word
           Return join(words, " ")
```

Java Implementation

```
class Solution {
           public String reverseWords(String s) {
               String[] words = s.split(" ");
               StringBuilder result = new StringBuilder();

                 for (int i = 0; i < words.length; i++) {
                     String reversed = new
StringBuilder(words[i]).reverse().toString();
                     result.append(reversed);
                     if (i < words.length - 1) {
                         result.append(" ");
                     }
                 }
                 return result.toString();
             }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

String Algorithms Summary

Problem	Type	Time	Space	Key Technique
Remove Outer Parentheses	Easy	O(n)	O(n)	Counter tracking
Reverse Words	Easy	O(n)	O(n)	Split and reverse
Largest Odd Number	Easy	O(n)	O(1)	Right-to-left scan
Longest Common Prefix	Easy	O(n*m)	O(1)	Compare first string
Isomorphic Strings	Easy	O(n)	O(1)	Two hash maps
Rotation Check	Easy	O(n)	O(n)	Check substring in s+s
Anagram Check	Easy	O(n)	O(1)	Frequency array
Sort by Frequency	Medium	O(n log n)	O(n)	Frequency + sorting
Max Nesting Depth	Medium	O(n)	O(1)	Counter tracking
Roman to/from Integer	Medium	O(n)	O(1)	Greedy mapping
Implement Atoi	Medium	O(n)	O(1)	Handle edge cases
Count Substrings	Medium	$O(n^2)$	O(1)	Sliding window
Longest Palindrome	Medium	$O(n^2)$	O(1)	Center expansion
Sum of Beauty	Medium	$O(n^2)$	O(1)	Frequency tracking
Reverse Each Word	Medium	O(n)	O(n)	Split and reverse